

2008

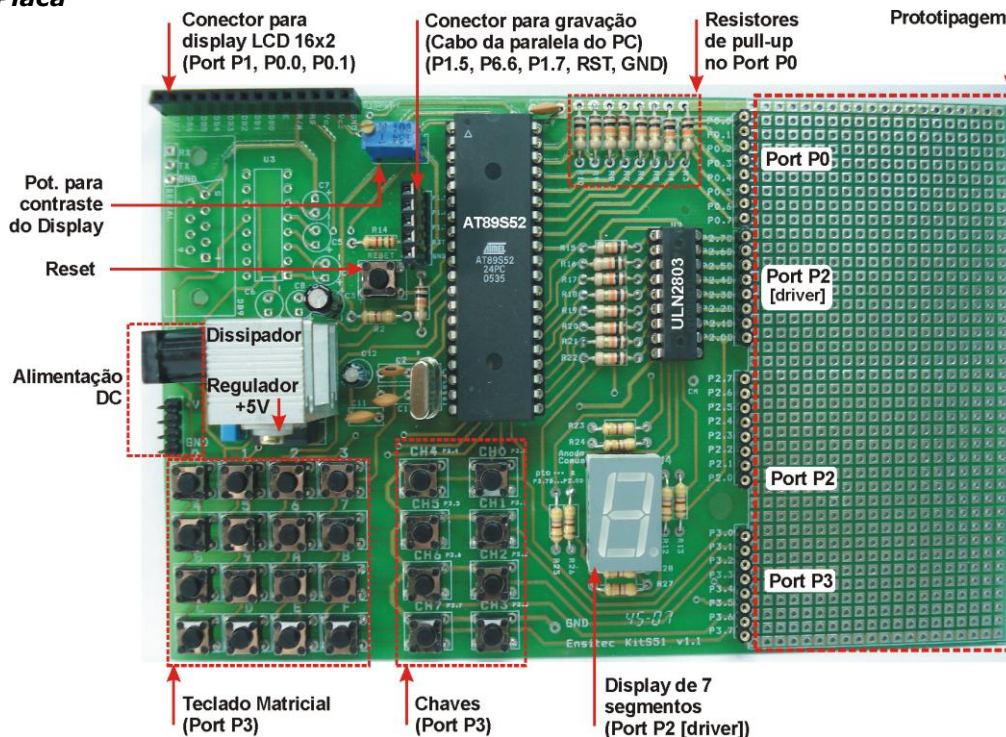
KIT S51

Kit didático para microcontrolador AT89S51/52

Requisitos

- Placa de circuito impresso "KIT S51 V1.1".
- Componentes para montagem da placa.
- Cabo para gravação.
- Fonte DC.
- Aplicativo "RIDE" para 8051, no qual serão desenvolvidos os programas a serem gravados no microcontrolador.
- Aplicativo para gravação "Aec_isp.exe" (DOS).

1. Placa



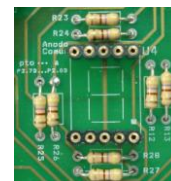
Lista de componentes:

- 17 resistores 10k Ω
- 9 resistores 470 Ω
- 1 resistor 1k5 Ω
- 1 trimpot multivoltas 10k Ω vertical
- 2 capacitores cerâmicos 100nF
- 2 capacitores eletrolíticos 10uF x 16V ou 25V
- 2 capacitores cerâmicos 33pF
- 1 capacitor poliéster 330nF
- 1 display de 7 segmentos anodo comum vermelho
- 1 cristal 11,059200 MHz
- 1 CI AT89S51 ou AT89S52
- 1 CI ULN2803
- 1 7805 (regulador de tensão)
- 25 micro-switch NA para PCI (ver foto da placa)
- Barra de pinos 180° [4 pinos]
- Barra de pinos fêmea para PCI (mini modul para PCI) [19 pinos]
- Barra de pinos fêmea torneado [42 pinos]
- 1 soquete torneado 40 pinos
- 1 soquete torneado 18 pinos
- 1 display LCD 16 x 2 sem backlight

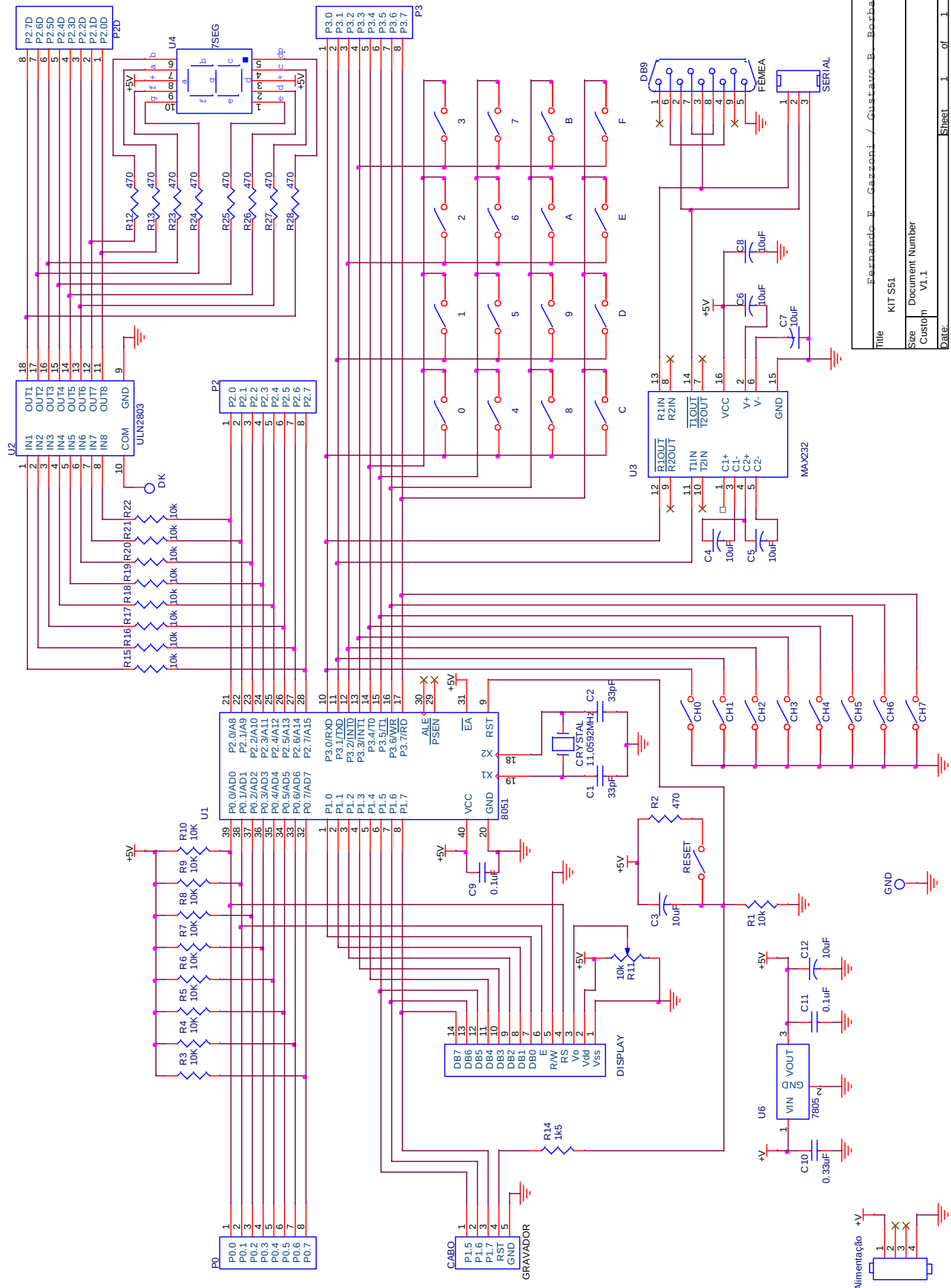
Com o display



Dica para montagem do display de 7 segmentos: soquete com barra de pinos fêmea.



2. Circuito



Title	Fernando E. Gazzoni / Gustavo B. Borba
Size	KIT S51
Document Number	-
Rev	-
Date:	Sheet 1 of 1

3. Cabo para gravação

Pinagem

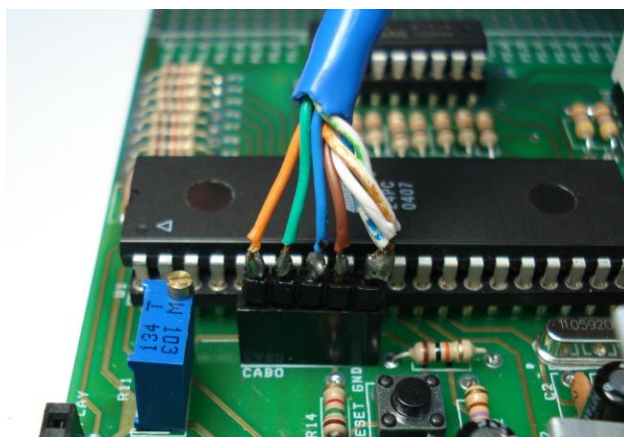
DB25 (porta paralela)	Conector "CABO" da placa
7 (D5)	P1.5
10 (ACK)	P1.6
8 (D6)	P1.7
6 (D4)	RST
18-25 (GND)	GND

Montagem

- Componentes: aproximadamente 60 cm. de cabo de rede (8 vias, 4 pares trançados); conector DB25 macho para cabo; barra de pinos 180° [5 pinos];
- Uma via (fio) de cada par trançado para cada sinal (D5, ACK, D6, D4).
- GNDs em um único ponto.



Cabo conectado na placa.



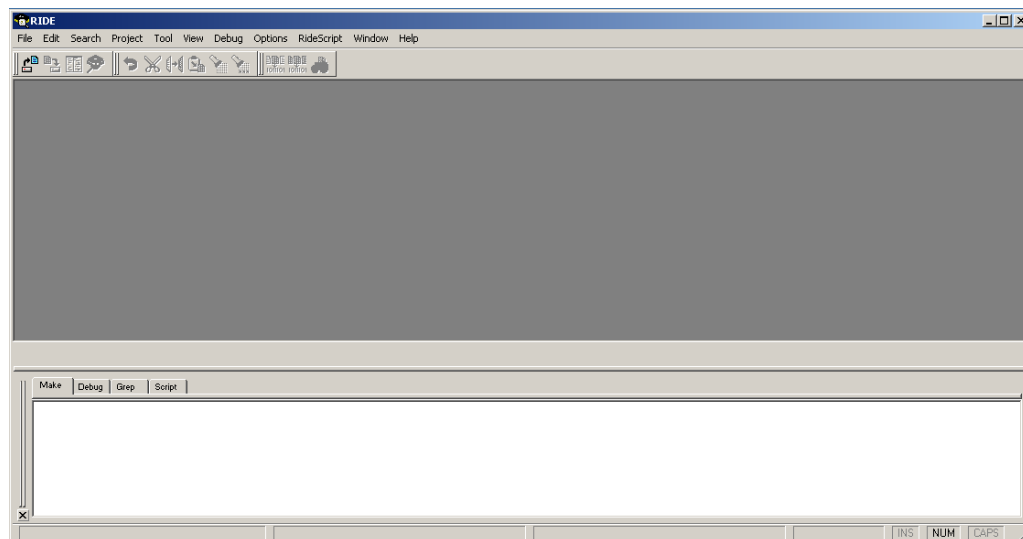
4. Aplicativos

- Para o desenvolvimento dos programas a serem gravados no microcontrolador:
RIDE - IDE Raisonance para 8051
 - 1º Faça o download em <http://www.raisonance.com/download/index.php>, clicando em [Eval51](#). É necessário um pequeno cadastro.
 - 2º Instale. O aplicativo chama-se RIDE. É uma versão de avaliação com todas as funcionalidades da versão completa, porém limitada a 4k de código.
- Para a gravação do programa no microcontrolador:
Aec_isp.exe
 Faça o Download em http://www.aec-electronics.co.nz/index.php?page_id=18. É necessário um pequeno cadastro.
 Roda em linha de comando (DOS) e grava os microcontroladores 8051 ATMEL da linha AT89S, modelos AT89S51/52/53/8252. Comunica-se com o microcontrolador pela porta paralela, utilizando o cabo descrito anteriormente.

5. Tutorial

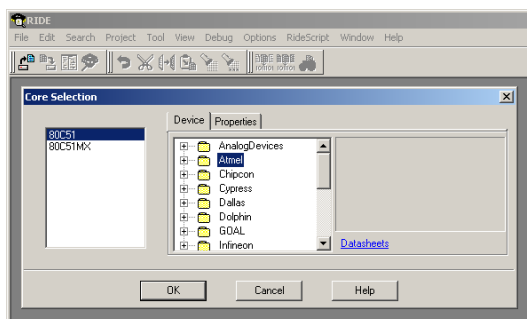
1º Programa a ser gravado no microcontrolador

Passo 1.
Abrir o
RIDE:

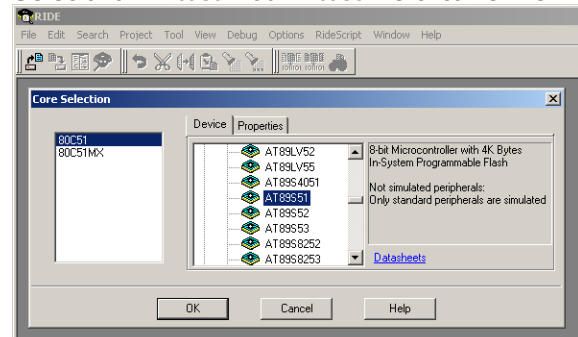


Passo 2. Selecionar a CPU: Options → TarGet

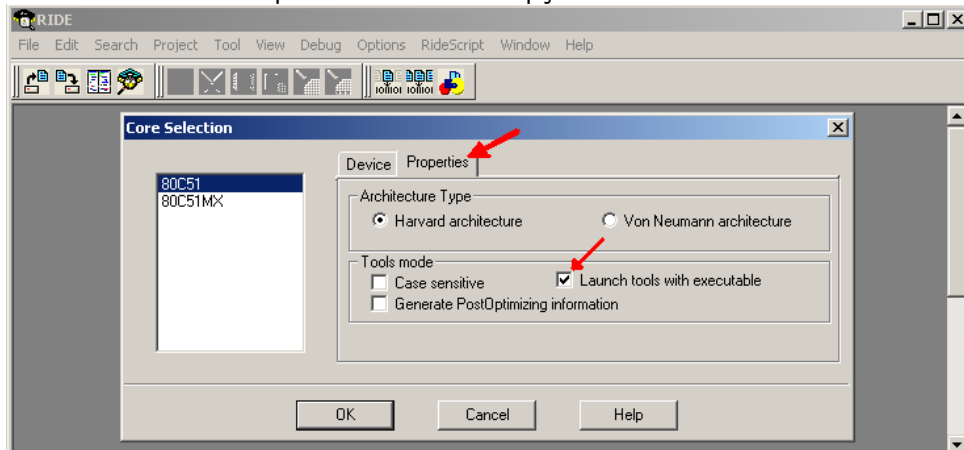
Selecionar '80C51' e abrir a pasta 'Atmel'



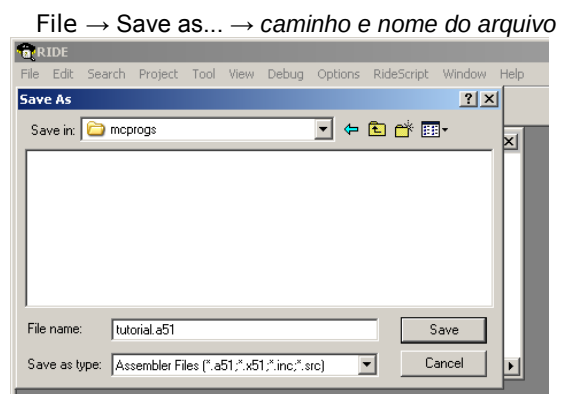
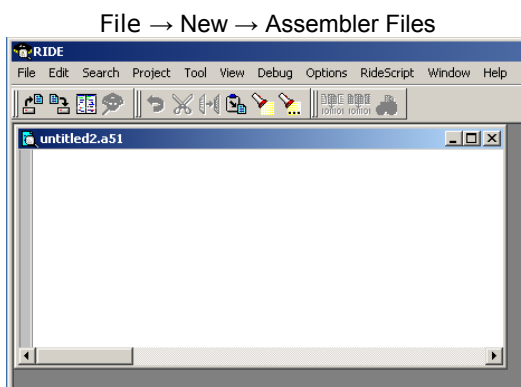
Selecionar 'AT89S51' ou 'AT89S52' e clicar em OK



Selecionar a aba 'Properties' e marcar a opção 'Launch tools with executable'



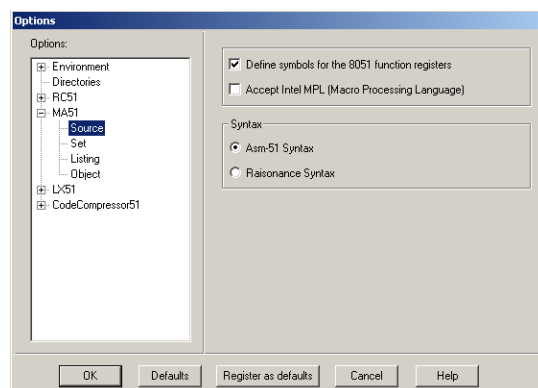
Passo 3. Criar e salvar um arquivo para o código fonte:



Passo 4. Configurar o Macro Assembler:
Options → Project → + MA51 → Source

Check no item
'Define Symbols for the
8051 function registers'
e clicar em OK.

(Esta configuração é armazenada, não
sendo mais necessária nas próximas
sessões de uso do RIDE)



Passo 5. Digitar o programa:

Este programa liga/desliga os segmentos do display de 7 segmentos a partir das chaves CH0...CH7.

Caracteres após o ';' são ignorados pelo assembler (comentários).

```
;KitS51 v1.1
;=====
;AUTOR:                                DATA:                                ARQUIVO: tutorial.a51
;CPU: AT89S51                          clk: 11,059200MHz
;=====
;O estado das chaves CH0...CH7 (P3.0...P3.7) é copiado para
;os segmentos a...pto do 7seg.
;-----
;chave CHn aberta (P3.n = 1) -> seg. on (P2.n = 1)
;chave CHn fechada (P3.n = 0) -> seg. off (P2.n = 0)
;=====

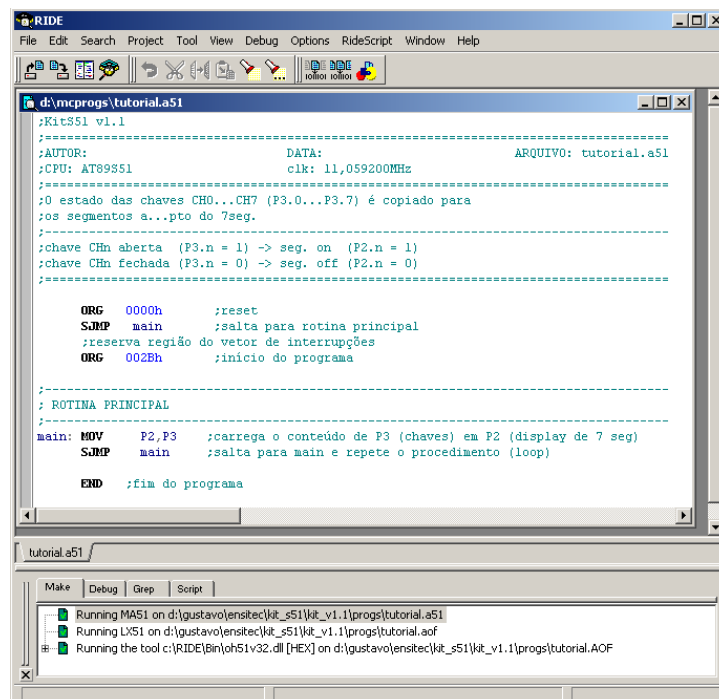
ORG 0000h      ;reset
SJMP main     ;salta para rotina principal
;reserva região do vetor de interrupções
ORG 002Bh      ;início do programa

;-----
; ROTINA PRINCIPAL
;-----
main: MOV      P2,P3      ;carrega o conteúdo de P3 (chaves) em P2 (display de 7 seg)
      SJMP     main      ;salta para main e repete o procedimento (loop)

END           ;fim do programa
```

Passo 6. Assemblar e linkar: Project → Make all (ou tecla 'F9')

O arquivo no formato hexa Intel foi gerado e encontra-se no mesmo diretório em que o programa foi salvo. Possui o mesmo nome do programa e extensão *.hex*. Neste caso, *tutorial.hex*.



2º Gravação e execução

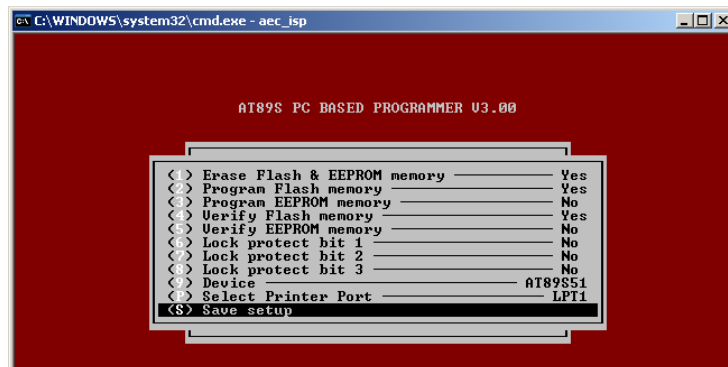
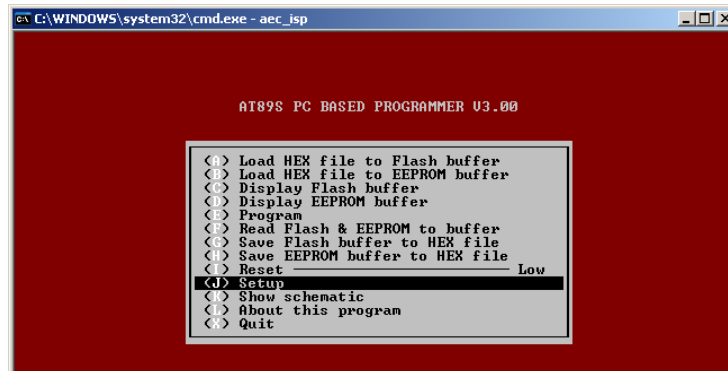
- A placa KIT S51 v1.1 deve estar alimentada.
- Conecte o cabo na paralela do PC e na placa KIT S51.
- Copie o programa *aec_isp.exe* no mesmo diretório do arquivo *tutorial.hex* e execute-o.
- Siga os passos:

Passo 1. Setup

Para escolha do modelo de microcontrolador e configuração dos lock bits.

Configure os parâmetros e saia desta tela fazendo um **Save setup**.

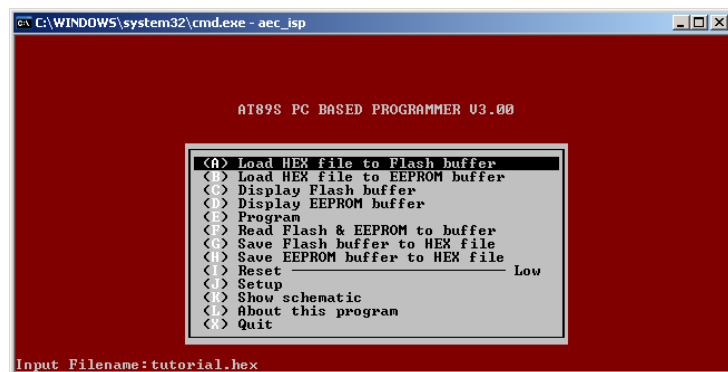
(Estas configurações são armazenadas em um arquivo chamado AECPRG.CFG.)



Passo 2. Load HEX file to Flash buffer

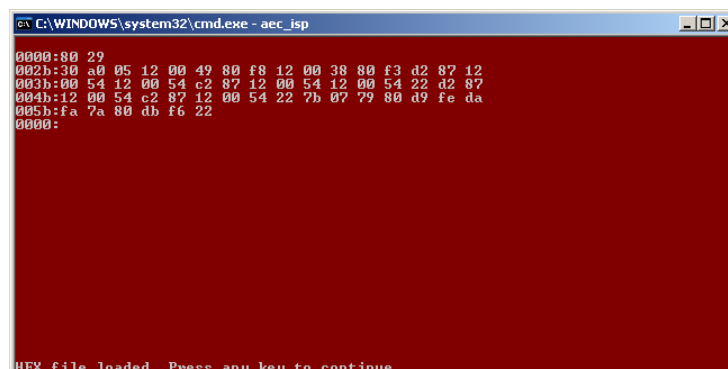
Para carregar o arquivo hexa Intel do seu programa.

Digite o caminho inteiro ou garanta que o .hex está no mesmo diretório do aec_isc.exe.



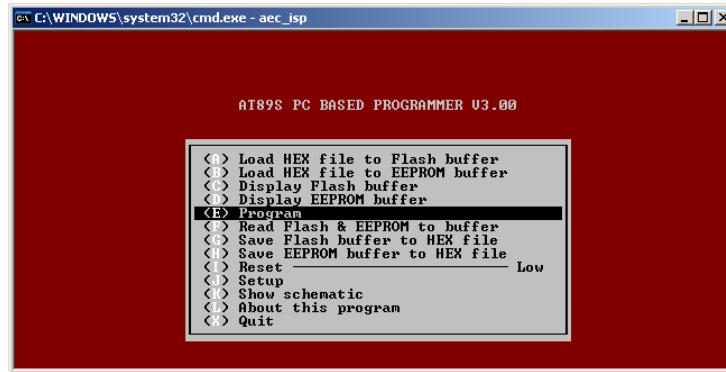
O buffer é mostrado, indicando que o programa foi carregado. (o conteúdo do buffer da figura ao lado é apenas ilustrativo. Não corresponde ao do programa tutorial.hex)

Pressione qualquer tecla para continuar.



Passo 3. Program

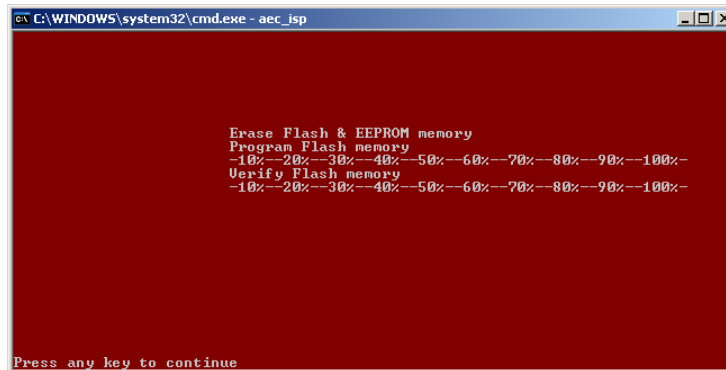
Para gravar.



OK!

Desconecte o cabo da placa e o programa estará rodando.

Caso tenha recebido uma mensagem de erro durante a gravação, revise a alimentação e o clock do microcontrolador e as conexões do cabo.



ATENÇÃO

Durante a gravação seu circuito está conectado diretamente na porta paralela do PC. Portanto, cuidado para não danificá-la! Para não haver riscos, durante o procedimento de gravação, é recomendável manter os pinos P1.5, P1.6 e P1.7 do AT89S desconectados do seu circuito, deixando-os em contato apenas com a porta paralela. Na placa KIT S51 estes pinos são compartilhados pelo barramento do display. Em condições normais, mesmo com o display conectado, a gravação ocorre normalmente e não há risco de danos à porta paralela.

6. Exemplos

Exemplo 1

```

;=====
;AUTOR: Gustavo B. Borba/Fernando E. Gazzoni   DATA: 10/2006           ARQUIVO: led.a51
;CPU: AT89S51                                clk: 11,059200MHz
;=====
;Chaves em P3.0 e P3.1 piscam os segmentos "a" e "d" do display de 7 seg.
;=====
;P3.0 = acionada -> Pisca os segmentos alternadamente
;P3.1 = acionada -> Pisca os segmentos em conjunto
;=====

ORG    00h
MOV    P2,#00;APAGA TODOS OS SEGMENTOS

;-----
; PROGRAMA PRINCIPAL --> TESTE DAS CHAVES
;-----

INICIO:
MOV     R0,P3
CJNE    R0,#11111110b,TESTE2 ;verifica se...
LCALL   CHAVE0; ...CH0 foi pressionada.
LJMP    INICIO
TESTE2:
CJNE    R0,#11111101b,INICIO ;verifica se...
LCALL   CHAVE1; ...CH1 foi pressionada.
LJMP    INICIO

```



```

;-----
; SUBROTINA PARA PISCAR OS SEGMENTOS ALTERNADAMENTE
;-----
CHAVE0:
    SETB P2.0      ;acende segmento a
    CLR P2.3       ;apaga segmento d
    LCALL ATRASO
    SETB P2.3      ;acende segmento d
    CLR P2.0       ;apaga segmento a
    LCALL ATRASO
    SETB P2.0      ;acende segmento a
    CLR P2.3       ;apaga segmento d
    LCALL ATRASO
    SETB P2.3      ;acende segmento d
    CLR P2.0       ;apaga segmento a
    LCALL ATRASO
    SETB P2.0      ;acende segmento a
    CLR P2.3       ;apaga segmento d
    LCALL ATRASO
    SETB P2.3      ;acende segmento d
    CLR P2.0       ;apaga segmento a
    LCALL ATRASO
    CLR P2.3       ;apaga segmento d
    RET

;-----
; SUBROTINA PARA PISCA OS SEGMENTOS EM CONJUNTO
;-----
CHAVE1:
    MOV P2,#09h ;acende segmento a e segmento d
    LCALL ATRASO
    MOV P2,#00h ;apaga segmento a e segmento d
    LCALL ATRASO
    MOV P2,#09h ;acende segmento a e segmento d
    LCALL ATRASO
    MOV P2,#00h ;apaga segmento a e segmento d
    LCALL ATRASO
    MOV P2,#09h ;acende segmento a e segmento d
    LCALL ATRASO
    MOV P2,#00h ;apaga segmento a e segmento d
    LCALL ATRASO
    RET

;-----
; SUBROTINA PARA GERAR ATRASO DE:
; (R3x24 + R3xR2x24 + R3xR2xR1x24)/freq clock
;-----
ATRASO:
    MOV R3,#7 ;gera atraso de...
    MOV R2,#255 ;... aproximadamente...
    MOV R1,#255 ;... 1 segundo
VOLTA:
    DJNZ R1,$
    MOV R1,#255
    DJNZ R2,VOLTA
    MOV R2,#255
    DJNZ R3,VOLTA
    RET

END

```

Exemplo 2

```

;=====
;AUTOR:Fernando E. Gazzoni/GUSTAVO B. BORBA DATA: 05/2008 ARQUIVO: lcd.a51 =
;CPU: AT89S51 clk: 11,059200MHz =
;=====
;Quatro chaves colocadas em P3.0, P3.1, P3.2 e P3.3 comandam o display LCD =
;=====
;P3.0 = acionada -> Apaga tudo e escreve "TESTE DE DISPLAY" na primeira linha =
;P3.1 = acionada -> Escreve "Kit S51" na segunda linha =
;P3.2 = acionada -> Apaga tudo e escreve uma frase nas duas linhas do display =
;P3.3 = acionada -> Apaga o que estiver escrito no display =
;=====

ORG 00h

MOV P2,#00 ;apaga todos os segmentos do display de 7 segmentos

;=====
; INICIO DO PROGRAMA PRINCIPAL =
;=====

LCALL INILCD

;-----
; LEITURA DAS CHAVES -
;-----

MAIN:
MOV A,P3
CJNE A,#11111110b,C1 ;se achave CH0 (em P3.0) for pressionada...
LCALL CH0 ;...chama subrotina CH0
JMP MAIN
C1:
CJNE A,#11111101b,C2 ;se a chave CH1 (em P3.1) for pressionada...
LCALL CH1 ;...chama a subrotina CH1
JMP MAIN
C2:
CJNE A,#11111011b,C3 ;se a chave CH2 (em P3.2) for pressionada...
LCALL CH2 ;... chama a subrotina CH2
JMP MAIN
C3:
CJNE A,#11110111b,MAIN ;se a chave CH3 (em P3.3) for pressionada...
LCALL CLEARLCD ;... apaga os dados escritos no LCD
JMP MAIN

;-----
; SUBROTINA DE TRATAMENTO DA CHAVE CH0 (EM P3.0) -
;-----
CH0:
LCALL CLEARLCD ;...apaga todos os caracteres escritos no display
MOV DPTR,#FRASE1
LCALL PRINTSTRING
RET

;-----
; SUBROTINA DE TRATAMENTO DA CHAVE CH1 (EM P3.1) -
;-----
CH1:
MOV B,#40h ;seleciona primeira coluna da segunda linha
LCALL POSCURSOR
MOV DPTR,#FRASE2
LCALL PRINTSTRING
RET

;-----
; SUBROTINA DE TRATAMENTO DA CHAVE CH2 (EM P3.2) -
;-----
CH2:
MOV DPTR,#FRASE3

```

```

        LCALL PRINTSTRINGALL
        RET

;-----
; FRASES A SEREM ESCRITAS NO DISPLAY
;-----
FRASE1:
        DB      'TESTE DE DISPLAY!\'
FRASE2:
        DB      'Kit S51\'
FRASE3:
        DB      'MICROCONTROLADORES/      ENSITEC\'

;=====
;=                      SUBROTINAS PARA O DISPLAY LCD                      =
;=====

;=====
;                      RESUMO DAS FUNCOES                      =
;=====
;      NOME      = ENTRADA =                      DESCRICAO                      =
;=====
;      INILCD      =      = Inicializacao do LCD                      =
;-----
;      CLEARLCD      =      = Apaga todos caracteres e coloca cursor no inicio =
;-----
;      POSCURSOR      =      B      = Seleciona a posicao do cursor no display      =
;-----
;      IWRITE      =      B      = Executa uma instrucao                      =
;-----
;      PRINTASCII      =      B      = Escreve um caractere ASCII no display      =
;-----
;      PRINTBCDL      =      B      = Escreve um digito BCD (menos significativo)      =
;-----
;      PRINTBCDHL      =      B      = Escreves dois digitos BCD                      =
;-----
;      PRINTEXAL      =      B      = Escreve um digito hexadecimal(menos significativo=
;-----
;      PRINTEXAHL      =      B      = Escreve dois digitos hexadecimal                      =
;-----
;      PRINTSTRING      =      DPTR      = Escreve frase em uma das linhas do display      =
;-----
;      PRINTSTRINGALL      =      DPTR      = Escreve frase nas duas linhas do display      =
;-----
;      SHIFTLCDR      =      B      = Desloca caracteres escritos no LCD para direita      =
;-----
;      SHIFTLCDL      =      B      = Desloca caracteres escritos no LCD para esquerda      =
;-----
;      DELAY      =      R6      = Gera atraso de R6 x 0,5ms                      =
;=====

;-----
; INICIALIZACAO DO DISPLAY
;-----
; ENTRADA: nenhuma
; SAIDA: nenhuma
;
; DESTROI: R1
; CHAMA: IWRITE, CLEARLCD, DELAY
;-----

INILCD:
        CLR      P0.1      ;desabilita a escrita no display
        MOV      R6,#50      ;gera atraso de 50x0,5ms = 25ms
        LCALL    DELAY
        MOV      B,#00110000b ;comando para inicializacao
        LCALL    IWRITE
        MOV      R6,#9      ;gera atraso de 9x0,5ms = 4,5ms
        LCALL    DELAY

```

```

        LCALL IWRITE ;repete instrucao
        MOV  R6,#1 ;gera atraso de 1x0,5ms = 0,5ms
        LCALL DELAY
        LCALL IWRITE ;repete instrucao

; Configuracao do usuario

        MOV  B,#00111000b
            ;|||_____ F = 0 -> matrizde 5x8 pontos
            ;||_____ N = 1-> 2 linhas
            ;|_____ DL = 1 -> 8 bits
        LCALL IWRITE

        MOV  B,#00001110b
            ;|||_____ B = 0 -> local do cursor nao pisca
            ;||_____ C = 1 -> cursor visivel
            ;|_____ D = 1 -> liga o display (aparece frase)
        LCALL IWRITE

        MOV  B,#00000110b
            ;||_____ SH=0 -> não desloca frase ao escrever caractere
            ;|_____ I/D = 1 -> cursor se move para direita
        LCALL IWRITE

        LCALL CLEARLCD
        RET

;-----
; SUBROTINA PARA LIMPAR O DISPLAY -
;-----
; Apaga todos os caracteres e posiciona o cursor no inicio do display -
;-----
; ENTRADA: nenhuma -
; SAIDA: nenhuma -
; -
; DESTROI: Caracteres armazenados da DDRAM, R6, A -
; CHAMA: IWRITE, POSCURSOR, DELAY -
;-----

CLEARLCD:
        PUSH B
        MOV  B,#00000001b ;Instrucao Clear display
        LCALL IWRITE
        MOV  R6,#3 ;atraso de 1,5ms -> tempo que a instrucao...
        LCALL DELAY ;...clear display leva para ser executada
        MOV  B,#0 ;posiciona o cursor no inicio do display -> endereco 00h
        LCALL POSCURSOR
        POP  B
        RET

;-----
; SUBROTINA PARA SELECIONAR A POSICAO DO CURSOR NO DISPLAY -
;-----
; ENTRADA: B --> endereco da DDRAM que o cursor sera posicionado -
; SAIDA: nenhuma -
; -
; DESTROI: A, B -
; CHAMA: IWRITE -
;-----

POSCURSOR:
        PUSH B ;armazena o valor de B na pilha
        MOV  A,B
        ORL  A,#10000000b
        MOV  B,A
        LCALL IWRITE
        POP  B ;restaura o valor de B
        RET

```

```

;-----
; SUBROTINA PARA EXECUTAR INSTRUcoes NO DISPLAY -
;-----
; ENTRADA: B --> instrucao a ser executada -
; SAIDA: nenhuma -
; -
; DESTROI: R7 -
; CHAMA: runT0 -
;-----

IWRITE:
    MOV    P1,B
    CLR    P0.0 ;seleciona RS = 0 (instrucao)
    SETB   P0.1 ;habilita escrita
    MOV    R7,#18 ;aguarda 39us para...
    DJNZ   R7,$ ;executar o comando
    CLR    P0.1 ;desabilita escrita
    RET

;-----
; SUBROTINA PARA ESCRIVER CARACTER ASCII NO DISPLAY -
;-----
; ENTRADA: B --> caractere (em ASCII) a ser escrito no display -
; SAIDA: nenhuma -
; -
; DESTROI: R7 -
; CHAMA: runT0 -
;-----

PRINTASCII:
    MOV    P1,B
    SETB   P0.0 ;seleciona RS = 1 (dado)
    SETB   P0.1 ;habilita escrita
    MOV    R7,#20 ;aguarda 43us para...
    DJNZ   R7,$ ;... realizar a escrita
    CLR    P0.1 ;desabilita escrita
    RET

;-----
; SUBROTINA PARA ESCRIVER DIGITO BCD MENOS SIGNIFICATIVO -
;-----
; Sero escritos caracteres indesejados se o numero nao estiver no formato BCD-
;-----
; ENTRADA: B --> numero BCD -
; SAIDA: nenhuma -
; -
; DESTROI: A -
; CHAMA: PRINTASCII -
;-----

PRINTBCDL:
    PUSH   B ;armazena conteudo de B na pilha
    MOV    A,B
    ANL    A,#00001111b ;usa apenas 4bits menos significativo
    ADD    A,#30h ;converte resultado para ASCII
    MOV    B,A
    LCALL  PRINTASCII
    POP    B ;restaura conteudo de B da pilha
    RET

```

```

; SUBROTINA PARA ESCREVER DOIS DIGITOS BCD -
;-----
; Sero escritos caracteres indesejados se o numero nao estiver no formato BCD-
;-----
; ENTRADA: B --> numero BCD -
; SAIDA: nenhuma -
; -
; DESTROI: A, B -
; CHAMA: PRINTASCII -
;-----
PRINTBCDHL:
    PUSH B ;armazena conteudo de B na pilha
    MOV A,B
    ANL A,#11110000b ;separa 4 bits mais significativos
    SWAP A ;troca nibble menos significativo por mais significativo
    ADD A,#30h ;converte para ASCII
    MOV B,A ;restaura conteudo de B da pilha
    LCALL PRINTASCII
    POP B
    MOV A,B
    ANL A,#00001111b ;separa 4 bits menos significativos
    ADD A,#30h ;converte para ASCII
    MOV B,A
    LCALL PRINTASCII
    RET

;-----
; SUBROTINA PARA ESCREVER DIGITO HEXADECIMAL MENOS SIGNIFICATIVO -
;-----
; ENTRADA: B --> numero HEXA -
; SAIDA: nenhuma -
; -
; DESTROI: A, DPTR -
; CHAMA: PRINTASCII -
;-----
PRINTHEXAL:
    PUSH B ;armazena conteudo de B na pilha
    MOV DPTR,#TABHEXA
    MOV A,B
    ANL A,#00001111b ;separa nibble menos significativo
    MOVC A,@A+DPTR ;converte caractere para ASCII
    MOV B,A
    LCALL PRINTASCII
    POP B ;restaura conteudo de B da pilha
    RET

;-----
; SUBROTINA PARA ESCREVER DOIS DIGITOS HEXADECIMAIS -
;-----
; ENTRADA: B --> numero HEXA -
; SAIDA: nenhuma -
; -
; DESTROI: A, DPTR -
; CHAMA: PRINTASCII -
;-----
PRINTHEXABL:
    PUSH B ;armazena conteudo de B na pilha
    MOV DPTR,#TABHEXA
    MOV A,B
    ANL A,#11110000b ;separa nibble mais significativo
    SWAP A ;troca nibble menos significativo por mais significativo
    MOVC A,@A+DPTR ;converte caractere para ASCII
    MOV B,A
    LCALL PRINTASCII
    POP B ;restaura conteudo de B da pilha
    MOV A,B
    ANL A,#00001111b ;separa nibble menos significativo
    MOVC A,@A+DPTR ;converte caractere para ASCII
    MOV B,A
    LCALL PRINTASCII
    RET

```



```

TABHEXA:
    DB      '0123456789ABCDEF'

;-----
; SUBROTINA PARA ESCREVER FRASE EM APENAS UMA LINHA DO DISPLAY -
;-----
; ENTRADA: DPTR --> frase a ser escrita -
; SAIDA: nenhuma -
; -
; DESTROI: A, R0 -
; CHAMA: PRINTASCII -
;-----
; Caractere especial: '\' -> fim da frase -
; -
; Nao verifica se a frase cabe no display -
; Precisa ser colocado o caractere '\' no final da frase -
;-----

PRINTSTRING:
    PUSH    B ;armazena conteudo de B na pilha
    MOV     R0,#0 ;inicia a escrita a partir da primeira letra da frase

PRINTS:
    MOV     A,R0 ;converte o caractere hexadecimal...
    MOVC    A,@A+DPTR ;... em ASCII
    MOV     B,A

    CJNE    A,'#\'',WRITE ;encontrou o caractere '\' ?
    POP     B              ;sim -> para de escrever
    RET

WRITE:
    LCALL   PRINTASCII
    INC     R0
    JMP     PRINTS

;-----
; SUBROTINA PARA ESCREVER FRASES NAS DUAS LINHAS DO DISPLAY -
;-----
; ENTRADA: DPTR --> frase a ser escrita -
; SAIDA: nenhuma -
; -
; DESTROI: A, R0 -
; CHAMA: PRINTASCII -
;-----
; Caracteres especiais: '/' -> quebra de linha (da primeira para a segunda) -
;                        '\' -> fim da frase -
; -
; Nao verifica se a frase cabe no display -
; Precisa ser colocado o caractere '\' no final da frase -
;-----

PRINTSTRINGALL:
    PUSH    B ;armazena conteudo de B na pilha
    LCALL   CLEARLCD
    MOV     R0,#0 ;inicia a escrita a partir da primeira letra da frase

PRINTSALL:
    MOV     A,R0 ;converte o caractere hexadecimal...
    MOVC    A,@A+DPTR ;... em ASCII
    MOV     B,A

    CJNE    A,'#\'',FIMSTR ;encontrou o caractere '/' ?
    MOV     B,#40h ;sim -> comeca a escrever na segunda linha
    LCALL   POSCURSOR
    INC     R0
    JMP     PRINTSALL

WRITEALL:
    LCALL   PRINTASCII
    INC     R0

```

```

                JMP     PRINTSALL

FIMSTR:
                CJNE   A,#'\',WRITEALL ;encontrou o caractere '\'?
                POP    B                ;sim -> para de escrever
                RET

;-----
; SUBROTINA PARA DESLOCAR A FRASE A DIREITA -
;-----
; Desloca para a direita todos os caracteres ja escritos no display -
;-----
; ENTRADA: nenhuma -
; SAIDA: nenhuma -
; -
; DESTROI: -
; CHAMA: IWRITE -
;-----

SHIFTLCDR:
                PUSH   B ;armazena conteudo de B na pilha
                MOV    B,#00011100b
                LCALL  IWRITE
                POP    B ;restaura conteudo de B na pilha
                RET

;-----
; SUBROTINA PARA DESLOCAR A FRASE A ESQUERDA -
;-----
; Desloca para a esquerda todos os caracteres ja escritos no display -
;-----
; ENTRADA: nenhuma -
; SAIDA: nenhuma -
; -
; DESTROI: B -
; CHAMA: IWRITE -
;-----

SHIFTLCDL:
                PUSH   B ;armazena conteudo de B na pilha
                MOV    B,#00011100b
                LCALL  IWRITE
                POP    B ;restaura conteudo de B na pilha
                RET

;-----
; SUBROTINA PARA GERAR ATRASO DE R1 x 0,5ms -
;-----
; ENTRADA: R6 --> valor a ser multiplicado pela base de tempo -
; -
; SAIDA: nenhuma -
; -
; DESTROI: R7, R6 -
;-----

DELAY:
                MOV    R7,#228
                DJNZ   R7,$
                DJNZ   R6,DELAY
                RET

                END

```

Exemplo 3

```
-----  
le:  
    JB     P3.3,le ;verifica se a chave em P3.3 foi acionada  
    CLR     DESLIGA ;zera o flag de parada do alarme  
    LCALL  alarme ;chama a subrotina para tocar o alarme  
    SJMP   le  
  
-----  
;ISR DA INTERRUPCAO EXTERNA 0  
-----  
  
int_ext0:  
    SETB    DESLIGA ;seta o flag de parada do alarme  
    RETI  
  
-----  
; SUBROTINA QUE COMPLEMENTA P2.7 COM CICLO ATIVO DE 50% USANDO Timer 0  
; OSCILACAO COM FREQ = 262Hz  
-----  
; ENTRADA: nenhuma  
; SAIDA: nenhuma  
; DESTROI: nada  
-----  
  
alarme:  
    MOV     TH0,#0F9h  
    MOV     TL0,#1Eh  
    SETB    TR0 ;dispara timer  
    JNB     TF0,$ ;preso  
    CLR     TR0 ;stop timer  
    CLR     TF0 ;zera flag overflow  
    CPL     P2.7 ;complementa P2.0  
    JNB     DESLIGA,alarme ;sai se o flag de parada estiver setado  
    RET  
  
END
```

Exemplo 4

```

;=====
;= AUTOR: FERNANDO GAZZONI/GUSTAVO BORBA      DATA:05/2008      ARQUIVO:tec.a51 =
;= CPU: AT89S51                               clk: 11,059200MHz      =
;=====
;=          Teclado matricial: escreve o valor da tecla no display LCD      =
;-----
;=          Usa as funcoes do display LCD                                  =
;=====

ORG 00h          ;reset
JMP INI          ;salta para rotina principal
;reserva região do vetor de interrupções
ORG 0050h        ;início do programa

;-----
;          INICIALIZACAO DO DISPLAY                                     -
;-----

INI:
MOV P2,#00 ;apaga display 7 segmentos
LCALL INILCD
LCALL CLEARLCD

;-----
; PROGRAMA PRINCIPAL                                               -
;-----

MAIN:
LCALL IN_HEX ;chama subrotina para ler teclado
MOV B,#07 ;escreve a partir do endereco...
LCALL POSCURSOR ;... 07 do LCD
MOV B,R6 ;copia o valor da tecla pressionada...
LCALL PRINTHEXAL ;... e mostra no LCD
SJMP MAIN

;=====
;=          SUBROTINAS PARA LEITURA DO TECLADO MATRICIAL          =
;=====

;-----
; SUBROTINA PARA DEBOUNCE DA LEITURA DO TECLADO MATRICIAL      -
; LIVRO: The 8051 Microcontroller      AUTOR: MacKenzie, I. Scott -
;-----
; ENTRADA: nenhuma                                             -
; SAIDA: nenhuma                                              -
; DESTROI: R4, R6                                             -
; CHAMA: READ                                                  -
;                                                                -
; USA DEBOUNCE DE 50 REPETICOES                                -
;-----

IN_HEX:
MOV R4,#50 ;contador para debounce
RETURN:
CALL READ ;chave foi pressionada?
JNC IN_HEX ;nao -> verifica novamente
DJNZ R4,RETURN ;sim -> repete 50x
PUSH ACC ;armazena na pilha o codigo hexa da tecla apertada
RETURN2:
MOV R4,#50 ;aguarda a chave ser solta
RETURN3:
CALL READ ;chave continua pressionada?
JC RETURN2 ;sim -> verifica novamente
DJNZ R4,RETURN3 ;nao -> repete 50x
POP ACC ;recupera o codigo hexa da chave e...
MOV R6,A ;... salva em R6
RET

;-----
; SUBROTINA PARA LEITURA E DECODIFICACAO DO TECLADO MATRICIAL -

```

```

; LIVRO: The 8051 Microcontroller      AUTOR: MacKenzie, I. Scott      -
;-----
; ENTRADA: chaves colocadas no port3      -
; SAIDA: DISP -> codigo hexadecimal da chave que foi apertada      -
;-----
; DESTROI: A, R5, R6 e R7      -
;-----

READ:
    MOV    A,#0FEh ;inicia na coluna 0
    MOV    R6,#4 ;R6 = contador

TEST_AGAIN:
    MOV    P3,A
    MOV    R7,A ;guarda o conteudo do acumulador
    MOV    A,P3 ;le port3
    ANL    A,#0F0h ;usa apenas os valores das linhas
    CJNE   A,#0F0h,CH_PRESS ;alguma linha esta ativa?
    MOV    A,R7 ;nao -> move para a proxima coluna
    RL     A
    DJNZ   R6,TEST_AGAIN ;nenhuma chave pressionada?
    CLR    C ;retorna com C=0
    SJMP   FIN

CH_PRESS:
    MOV    R7,A ;salva o codigo da coluna em R7
    MOV    A,#4 ;prepara o calculo do peso da coluna
    CLR    C
    SUBB   A,R6 ;4-R6 = peso da coluna
    MOV    R6,A ;guarda o resultado em R6
    MOV    A,R7 ;restaura a busca para A
    SWAP   A ;coloca o resultado da busca no nibble LS
    MOV    R5,#4 ;usa R5 como um contador

AGAIN:
    RRC    A ;rotaciona ate encontrar o bit 0
    JNC    HEX ;sai quando C=0
    INC    R6 ;adiciona 4...
    INC    R6 ;...
    INC    R6 ;...
    INC    R6 ;... ate encontrar a coluna
    DJNZ   R5,AGAIN

HEX:
    SETB   C ;C=1 -> chave pressionada
    MOV    A,R6 ;codigo hex salvo em A

FIN:
    RET

;=====
;=                SUBROTINAS PARA O DISPLAY LCD                =
;=====

;-----
; INICIALIZACAO DO DISPLAY      -
;-----
; ENTRADA: nenhuma      -
; SAIDA: nenhuma      -
;-----
; DESTROI: R6      -
; CHAMA: IWRITE, CLEARLCD, DELAY      -
;-----

INILCD:
    CLR    P0.1 ;desabilita a escrita no display
    MOV    R6,#50 ;gera atraso de 50x0,5ms = 25ms
    LCALL  DELAY
    MOV    B,#00110000b ;comando para inicializacao
    LCALL  IWRITE
    MOV    R6,#9 ;gera atraso de 9x0,5ms = 4,5ms
    LCALL  DELAY
    LCALL  IWRITE ;repete instrucao
    MOV    R6,#1 ;gera atraso de 1x0,5ms = 0,5ms
    LCALL  DELAY
    LCALL  IWRITE ;repete instrucao

```

```
; Configuracao do usuario
```

```
MOV    B,#00111000b
        ;|||_____ F = 0 -> matrizde 5x8 pontos
        ;||_____ N = 1-> 2 linhas
        ;|_____ DL = 1 -> 8 bits

LCALL  IWRITE

MOV    B,#00001110b
        ;|||_____ B = 0 -> local do cursor nao pisca
        ;||_____ C = 1 -> cursor visivel
        ;|_____ D = 1 -> liga o display (aparece frase)

LCALL  IWRITE

MOV    B,#00000110b
        ;||_____ SH=0 -> não desloca frase ao escrever caractere
        ;|_____ I/D = 1 -> cursor se move para direita

LCALL  IWRITE

MOV    B,#00000010b ;return home -> retorna o cursor para a...
LCALL  IWRITE ;...primeira linha e primeira coluna

LCALL  CLEARLCD

RET
```

```
;-----
; SUBROTINA PARA LIMPAR O DISPLAY -
;-----
; Apaga todos os caracteres e posiciona o cursor no inicio do display -
;-----
; ENTRADA: nenhuma -
; SAIDA: nenhuma -
;
; DESTROI: Caracteres armazenados da DDRAM, R6, A -
; CHAMA: IWRITE, POSCURSOR, DELAY -
;-----
```

```
CLEARLCD:
```

```
PUSH    B
MOV     B,#00000001b ;Instrucao Clear display
LCALL  IWRITE
MOV     R6,#3 ;atraso de 1,5ms -> tempo que a instrucao...
LCALL  DELAY ;...clear display leva para ser executada
MOV     B,#0 ;posiciona o cursor no inicio do display -> endereco 00h
LCALL  POSCURSOR
POP     B
RET
```

```
;-----
; SUBROTINA PARA SELECIONAR A POSICAO DO CURSOR NO DISPLAY -
;-----
; ENTRADA: B --> endereco da DDRAM que o cursor sera posicionado -
; SAIDA: nenhuma -
;
; DESTROI: A -
; CHAMA: IWRITE -
;-----
```

```
POSCURSOR:
```

```
PUSH    B
MOV     A,B
ORL     A,#10000000b
MOV     B,A
LCALL  IWRITE
POP     B
RET
```

```
;-----
```

```

; SUBROTINA PARA EXECUTAR INSTRUÇOES NO DISPLAY -
;-----
; ENTRADA: B --> instrucao a ser executada -
; SAIDA: nenhuma -
; -
; DESTROI: R7 -
; CHAMA: runT0 -
;-----

IWRITE:
    MOV    P1,B
    CLR    P0.0 ;seleciona RS = 0 (instrucao)
    SETB   P0.1 ;habilita escrita
    MOV    R7,#18 ;aguarda 39us para...
    DJNZ   R7,$ ;executar o comando
    CLR    P0.1 ;desabilita escrita
    RET

;-----
; SUBROTINA PARA ESCREVER CARACTER ASCII NO DISPLAY -
;-----
; ENTRADA: B --> caractere (em ASCII) a ser escrito no display -
; SAIDA: nenhuma -
; -
; DESTROI: R7 -
; CHAMA: runT0 -
;-----

PRINTASCII:
    MOV    P1,B
    SETB   P0.0 ;seleciona RS = 1 (dado)
    SETB   P0.1 ;habilita escrita
    MOV    R7,#20 ;aguarda 43us para...
    DJNZ   R7,$ ;... realizar a escrita
    CLR    P0.1 ;desabilita escrita
    RET

;-----
; SUBROTINA PARA ESCREVER DIGITO BCD MENOS SIGNIFICATIVO -
;-----
; Sero escritos caracteres indesejados se o numero nao estiver no formato BCD-
;-----
; ENTRADA: B --> numero BCD -
; SAIDA: nenhuma -
; -
; DESTROI: A -
; CHAMA: PRINTASCII -
;-----

PRINTBCDL:
    PUSH   B ;armazena conteudo de B na pilha
    MOV    A,B
    ANL    A,#00001111b ;usa apenas 4bits menos significativo
    ADD    A,#30h ;converte resultado para ASCII
    MOV    B,A
    LCALL  PRINTASCII
    POP    B ;restaura conteudo de B da pilha
    RET

;-----
; SUBROTINA PARA ESCREVER DOIS DIGITOS BCD -
;-----
; Sero escritos caracteres indesejados se o numero nao estiver no formato BCD-
;-----
; ENTRADA: B --> numero BCD -
; SAIDA: nenhuma -
; -
; DESTROI: A, B -
; CHAMA: PRINTASCII -
;-----

```


PRINTBCDHL:

```

    PUSH    B ;armazena conteudo de B na pilha
    MOV     A,B
    ANL     A,#11110000b ;separa 4 bits mais significativos
    SWAP    A ;troca nibble menos significativo por mais significativo
    ADD     A,#30h ;converte para ASCII
    MOV     B,A ;restaura conteudo de B da pilha
    LCALL   PRINTASCII
    POP     B
    MOV     A,B
    ANL     A,#00001111b ;separa 4 bits menos significativos
    ADD     A,#30h ;converte para ASCII
    MOV     B,A
    LCALL   PRINTASCII
    RET

```

```

;-----
; SUBROTINA PARA ESCREVER DIGITO HEXADECIMAL MENOS SIGNIFICATIVO -
;-----
; ENTRADA: B --> numero HEXA -
; SAIDA: nenhuma -
;
; DESTROI: A, DPTR -
; CHAMA: PRINTASCII -
;-----

```

PRINTHEXAL:

```

    PUSH    B ;armazena conteudo de B na pilha
    MOV     DPTR,#TABHEXA
    MOV     A,B
    ANL     A,#00001111b ;separa nibble menos significativo
    MOVC    A,@A+DPTR ;converte caractere para ASCII
    MOV     B,A
    LCALL   PRINTASCII
    POP     B ;restaura conteudo de B da pilha
    RET

```

```

;-----
; SUBROTINA PARA ESCREVER DOIS DIGITOS HEXADECIMAIS -
;-----
; ENTRADA: B --> numero HEXA -
; SAIDA: nenhuma -
;
; DESTROI: A, B, DPTR -
; CHAMA: PRINTASCII -
;-----

```

PRINTHEXABL:

```

    PUSH    B ;armazena conteudo de B na pilha
    MOV     DPTR,#TABHEXA
    MOV     A,B
    ANL     A,#11110000b ;separa nibble mais significativo
    SWAP    A ;troca nibble menos significativo por mais significativo
    MOVC    A,@A+DPTR ;converte caractere para ASCII
    MOV     B,A
    LCALL   PRINTASCII
    POP     B ;restaura conteudo de B da pilha
    MOV     A,B
    ANL     A,#00001111b ;separa nibble menos significativo
    MOVC    A,@A+DPTR ;converte caractere para ASCII
    MOV     B,A
    LCALL   PRINTASCII
    RET

```

TABHEXA:

```

    DB      '0123456789ABCDEF'

```

```

;-----
; SUBROTINA PARA ESCREVER FRASE EM APENAS UMA LINHA DO DISPLAY -
;-----
; ENTRADA: DPTR --> frase a ser escrita -
; SAIDA: nenhuma -
; -
; DESTROI: A, R7 -
; CHAMA: PRINTASCII -
;-----
; Caractere especial: '\' -> fim da frase -
; -
; Nao verifica se a frase cabe no display -
; Precisa ser colocado o caractere '\' no final da frase -
;-----

PRINTSTRING:
    PUSH    B ;armazena conteudo de B na pilha
    MOV     R7,#0 ;inicia a escrita a partir da primeira letra da frase

PRINTS:
    MOV     A,R7 ;converte o caractere hexadecimal...
    MOVC    A,@A+DPTR ;... em ASCII
    MOV     B,A

    CJNE    A,#'\',WRITE ;encontrou o caractere '\' ?
    POP     B             ;sim -> para de escrever
    RET

WRITE:
    LCALL   PRINTASCII
    INC     R7
    JMP     PRINTS

;-----
; SUBROTINA PARA ESCREVER FRASES NAS DUAS LINHAS DO DISPLAY -
;-----
; ENTRADA: DPTR --> frase a ser escrita -
; SAIDA: nenhuma -
; -
; DESTROI: A, R7 -
; CHAMA: PRINTASCII -
;-----
; Caracteres especiais: '/' -> quebra de linha (da primeira para a segunda) -
;                          '\' -> fim da frase -
; -
; Nao verifica se a frase cabe no display -
; Precisa ser colocado o caractere '\' no final da frase -
;-----

PRINTSTRINGALL:
    PUSH    B ;armazena conteudo de B na pilha
    LCALL   CLEARLCD
    MOV     R7,#0 ;inicia a escrita a partir da primeira letra da frase

PRINTSALL:
    MOV     A,R7 ;converte o caractere hexadecimal...
    MOVC    A,@A+DPTR ;... em ASCII
    MOV     B,A

    CJNE    A,#'/',FIMSTR ;encontrou o caractere '/' ?
    MOV     B,#40h ;sim -> começa a escrever na segunda linha
    LCALL   POSCURSOR
    INC     R7
    JMP     PRINTSALL

WRITEALL:
    LCALL   PRINTASCII
    INC     R7
    JMP     PRINTSALL

FIMSTR:
    CJNE    A,#'\',WRITEALL ;encontrou o caractere '\' ?

```

```

        POP    B                ;sim -> para de escrever
        RET

;-----
; SUBROTINA PARA DESLOCAR A FRASE A DIREITA -
;-----
; Desloca para a direita todos os caracteres ja escritos no display -
;-----
; ENTRADA: nenhuma -
; SAIDA: nenhuma -
; -
; DESTROI: -
; CHAMA: IWRITE -
;-----

SHIFTLCDR:
        PUSH   B ;armazena conteudo de B na pilha
        MOV    B,#00011100b
        LCALL  IWRITE
        POP    B ;restaura conteudo de B na pilha
        RET

;-----
; SUBROTINA PARA DESLOCAR A FRASE A ESQUERDA -
;-----
; Desloca para a esquerda todos os caracteres ja escritos no display -
;-----
; ENTRADA: nenhuma -
; SAIDA: nenhuma -
; -
; DESTROI: B -
; CHAMA: IWRITE -
;-----

SHIFTLCDL:
        PUSH   B ;armazena conteudo de B na pilha
        MOV    B,#00011000b
        LCALL  IWRITE
        POP    B ;restaura conteudo de B na pilha
        RET

;-----
; SUBROTINA PARA GERAR ATRASO DE R1 x 0,5ms -
;-----
; ENTRADA: R6 --> valor a ser multiplicado pela base de tempo -
; -
; SAIDA: nenhuma -
; -
; DESTROI: R7, R6 -
;-----

DELAY:
        MOV     R7,#228
        DJNZ    R7,$
        DJNZ    R6,DELAY
        RET

        END

```

Exemplo 5

```

;=====
;= AUTOR: FERNANDO GAZZONI/GUSTAVO BORBA    DATA:05/2008    ARQUIVO:clock.a51  =
;= CPU: AT89S51                               clk: 11,059200MHz              =
;=====
;=                                     RELOGIO DIGITAL                                     =

```

```

;-----
;=                               USA AS FUNCOES DO LCD                               =
;-----
;CH0 --> Ajusta relógio                                                    =
;CH1 --> Inicia o relógio                                                  =
;                                                                           =
; --> Dentro do menu de ajuste:                                           =
;CH0 --> Confirma ajuste                                                  =
;CH1 --> Configura as horas                                              =
;CH2 --> Configura os minutos                                            =
;CH3 --> Zera o relógio (00:00:00)                                       =
;-----

;-----
;                               VARIAVEIS DO RELOGIO                               -
;-----

HORA      EQU    40h ;armazena as horas
MINUTO    EQU    41h ;armazena os minutos
SEGUNDO   EQU    42h ;armazena os segundos
TESTE     EQU    43h ;testa se passou 1seg para atualizar o relógio

;-----
;      INICIO DO PROGRAMA E CONFIGURACAO DA INTERRUPCAO DO TIMER 1      -
;-----

      ORG    0000h ; reset do sistema
      LJMP   INICIO

      ORG    001Bh ; interrupcao timer 1
      MOV    TH1,#4Ch ; recarrega TH1
      INC    TESTE
      MOV    A,TESTE
      CJNE   A,#20,FIM ;se teste = 20 x 50ms = 1 segundo...
      MOV    TESTE,#0 ;... reinicia variavel teste e...
      LCALL  RELOGIO ;... atualiza relógio

FIM:
      RETI

;-----
;      INICIALIZACAO: APENAS INT TIMER 1 HABILITADA, PRIORIDADE ALTA      -
;-----

      ORG    0050h

INICIO:

      MOV    IE,#00000000B ;desabilita todas as interrupcoes

      MOV    TCON,#00000000B
      ; ||||_____ TR0 = 0 -> timer 0 parado
      ; |||_____ TF0 = 0 -> flag de overflow do T/C 0 zerado
      ; ||_____ TR1 = 0 -> timer 0 parado
      ; |_____ TF1 = 0 -> flag de overflow do T/C 0 zerado

      MOV    TMOD,#00010000B
      ; ||||_____ M0 = 1 -> timer 1 no...
      ; |||_____ M1 = 0 -> ...modo 1: 16 bits
      ; ||_____ C/T' = 0 -> timer 1 na funcao timer
      ; |_____ GATE = 0 -> timer 1 controlado somente por TR0
      ;T/C 1 nao utilizado

      MOV    IP,#00001000b
      ; |_____ PT1 = 1 -> prioridade para a int do timer 1

      MOV    IE,#10001000b
      ; |_____ ET1 = 1 -> habilita a int do timer 1
      ; |_____ EA = 1 -> habilita todas as ints que...
      ; ...estiverem habilitadas

;-----
; SUBROTINA DE INICIALIZACAO DO DISPLAY                                     -
;-----

```

```

                LCALL  INILCD

;-----
;  PROGRAMA PRINCIPAL
;-----

                LCALL  ZERA

CHAVE0:
                MOV    A,P3
                CJNE   A,#11111110b,START ;AJUSTAR O RELOGIO?
                CLR    TR1

SOLTA:
                MOV    A,P3
                CJNE   A,#11111111b,SOLTA ;INICIAR O RELOGIO?
                LCALL  AJUSTE
                LJMP   CHAVE0

START:
                CJNE   A,#11111101b,CHAVE0 ;INICIAR O RELOGIO?
                LCALL  TIMER1
                LJMP   CHAVE0

;-----
;SUBROTINA PARA TRATAMENTO DA INTERRUPCAO EXTERNA 0
;-----

AJUSTE:
                MOV    R0,P3
                CJNE   R0,#11111110b,CHAVE1 ; terminou ajuste?
TERMINAAJUSTE:
                MOV    R0,P3
                CJNE   R0,#11111111b,TERMINAAJUSTE ; verifica se chave 0 foi solta
                RET

CHAVE1:
                CJNE   R0,#11111101b,CHAVE2 ; se chave 1 pressionada...
                LCALL  AJUSTEHORA                ; ...incrementa hora

SOLTAHORA:
                MOV    R0,P3
                CJNE   R0,#11111111b,SOLTAHORA ; verifica se chave 1 foi solta
                LJMP   AJUSTE

CHAVE2:
                CJNE   R0,#11111011b,CHAVE3      ; se chave 0 pressionada...
                LCALL  AJUSTEMIN                ; ...incrementa minuto

SOLTAMIN:
                MOV    R0,P3
                CJNE   R0,#11111111b,SOLTAMIN    ; verifica se chave 2 foi solta
                LJMP   AJUSTE

CHAVE3:
                CJNE   R0,#11110111b,AJUSTE ; se chave 2 pressionada...
                LCALL  ZERA                ; ... zera o relógio

SOLTAZERA:
                MOV    R0,P3
                CJNE   R0,#11111111b,SOLTAZERA ; verifica se chave 3 foi solta
                LJMP   AJUSTE

;-----
;AJUSTE DAS HORAS E MINUTOS
;-----

AJUSTEHORA:
                MOV    A,HORA      ;aumenta 1 hora e...
                ADD    A,#1        ;... ajusta o numero hexa ...
                DA      A          ; ...para o formato BCD
                MOV    HORA,A      ; atualiza a variavel hora
                CJNE   A,#24h,ATUALIZAHORA ;se hora = 24...
                MOV    HORA,#00h   ;... zera hora

ATUALIZAHORA:
                LCALL  HOR
                RET

```

```

AJUSTEMIN:
    MOV    A,MINUTO    ;aumenta 1 minuto e...
    ADD    A,#1        ;... ajusta o numero hexa ...
    DA     A           ; ...para o formato BCD
    MOV    MINUTO,A    ; atualiza a variavel minuto
    CJNE   A,#60h,ATUALIZAMIN ;se minuto = 60...
    MOV    MINUTO,#00h ; zera minuto

ATUALIZAMIN:
    LCALL  MIN
    RET

;-----
;TRTAMENTO DA INTERRUPCAO DO TIMER --> SUBROTINA DO RELOGIO
;-----

RELOGIO:
    MOV    A,SEGUNDO ;aumenta 1 segundo e...
    ADD    A,#1      ;... ajusta o numero hexa ...
    DA     A         ; ...para o formato BCD
    MOV    SEGUNDO,A ; atualiza a variavel segundo
    CJNE   A,#60h,SEG ;se segundo = 60...
    MOV    SEGUNDO,#00h ;... zera os segundos e aumenta os minutos

SOMAMIN:
    MOV    A,MINUTO    ;aumenta 1 minuto e...
    ADD    A,#1        ;... ajusta o numero hexa ...
    DA     A           ; ...para o formato BCD
    MOV    MINUTO,A    ; atualiza a variavel minuto
    CJNE   A,#60h,MIN  ;se minuto = 60...
    MOV    MINUTO,#00h ;... zera os minutos e aumenta as horas

SOMAHORA:
    MOV    A,HORA      ;aumenta 1 hora e...
    ADD    A,#1        ;... ajusta o numero hexa ...
    DA     A           ; ...para o formato BCD
    MOV    HORA,A      ; atualiza a variavel hora
    CJNE   A,#24h,HOR  ;se hora = 24: chama a rotina para...
    LCALL  ZERA        ;... colocar o relógio em 00:00:00
    RET

;-----
; ATUALIZACAO DO RELOGIO NO DISPLAY LCD
;-----

SEG:
    MOV    B,#0Ah      ;atualiza o valor...
    LCALL  POSCURSOR   ;... dos segundos...
    MOV    B,SEGUNDO   ;... no display LCD
    LCALL  PRINTBCDHL
    RET

MIN:
    MOV    B,#07h
    LCALL  POSCURSOR   ;atualiza o valor...
    MOV    B,MINUTO    ;... dos minutos...
    LCALL  PRINTBCDHL ;... no display LCD
    JMP    SEG

HOR:
    MOV    B,#04h
    LCALL  POSCURSOR   ;atualiza o valor...
    MOV    B,HORA      ;... das horas...
    LCALL  PRINTBCDHL ;... no display LCD
    JMP    MIN

;-----
; SUBROTINA PARA ZERAR O RELOGIO
;-----

ZERA:
    MOV    HORA,#00h    ;zera variavel hora
    MOV    MINUTO,#00h  ;zera variavel minuto
    MOV    SEGUNDO,#00h ;zera variavel segundo
    MOV    TESTE,#00h   ;zera variavel teste

    MOV    B,#04h      ;ajusta a posicao do cursor para o relógio...
    LCALL  POSCURSOR   ;... ficar no meio do display

```

```

MOV    DPTR,#MEIANOITE ;escreve a frase...
LCALL  PRINTSTRING      ;...meia noite no display
RET

MEIANOITE:
DB      '00:00:00\'

;-----
; SUBROTINA PARA LIGAR TIMER 1 COM ATRASO DE 50ms
;-----

TIMER1:
MOV     TH1,#4Ch ; fclk CPU = 11059200Hz -> 46.080 x 1,085us...
MOV     TL1,#00h ; ... base de tempo de 50ms
SETB    TR1      ; dispara timer
RET

;=====
;=                      SUBROTINAS PARA O DISPLAY LCD                      =
;=====

;-----
; INICIALIZACAO DO DISPLAY
;-----
; ENTRADA: nenhuma
; SAIDA: nenhuma
;
; DESTROI: R1
; CHAMA: IWRITE, CLEARLCD, DELAY
;-----

INILCD:
CLR     P0.1      ;desabilita a escrita no display
MOV     R6,#50    ;gera atraso de 50x0,5ms = 25ms
LCALL   DELAY
MOV     B,#00110000b ;comando para inicializacao
LCALL   IWRITE
MOV     R6,#9     ;gera atraso de 9x0,5ms = 4,5ms
LCALL   DELAY
LCALL   IWRITE ;repete instrucao
MOV     R6,#1     ;gera atraso de 1x0,5ms = 0,5ms
LCALL   DELAY
LCALL   IWRITE ;repete instrucao

; Configuracao do usuario

MOV     B,#00111000b
;|||_____ F = 0 -> matrizde 5x8 pontos
;||_____ N = 1-> 2 linhas
;|_____ DL = 1 -> 8 bits
LCALL   IWRITE

MOV     B,#00001110b
;|||_____ B = 0 -> local do cursor nao pisca
;||_____ C = 1 -> cursor visivel
;|_____ D = 1 -> liga o display (aparece frase)
LCALL   IWRITE

MOV     B,#00000110b
;||_____ SH=0 -> não desloca frase ao escrever caractere
;|_____ I/D = 1 -> cursor se move para direita
LCALL   IWRITE

LCALL   CLEARLCD
RET

;-----
; SUBROTINA PARA LIMPAR O DISPLAY
;-----
; Apaga todos os caracteres e posiciona o cursor no inicio do display
;-----
; ENTRADA: nenhuma

```

```

; SAIDA: nenhuma -
;
; DESTROI: Caracteres armazenados da DDRAM, R6, A -
; CHAMA: IWRITE, POSCURSOR, DELAY -
;-----

CLEARLCD:
    PUSH    B
    MOV     B,#00000001b ;Instrucao Clear display
    LCALL   IWRITE
    MOV     R6,#3 ;atraso de 1,5ms -> tempo que a instrucao...
    LCALL   DELAY ;...clear display leva para ser executada
    MOV     B,#0 ;posiciona o cursor no inicio do display -> endereco 00h
    LCALL   POSCURSOR
    POP     B
    RET

;-----
; SUBROTINA PARA SELECIONAR A POSICAO DO CURSOR NO DISPLAY -
;-----
; ENTRADA: B --> endereco da DDRAM que o cursor sera posicionado -
; SAIDA: nenhuma -
;
; DESTROI: A, B -
; CHAMA: IWRITE -
;-----

POSCURSOR:
    PUSH    B ;armazena o valor de B na pilha
    MOV     A,B
    ORL     A,#10000000b
    MOV     B,A
    LCALL   IWRITE
    POP     B ;restaura o valor de B
    RET

;-----
; SUBROTINA PARA EXECUTAR INSTRUcoes NO DISPLAY -
;-----
; ENTRADA: B --> instrucao a ser executada -
; SAIDA: nenhuma -
;
; DESTROI: R7 -
; CHAMA: runT0 -
;-----

IWRITE:
    MOV     P1,B
    CLR     P0.0 ;seleciona RS = 0 (instrucao)
    SETB    P0.1 ;habilita escrita
    MOV     R7,#18 ;aguarda 39us para...
    DJNZ    R7,$ ;executar o comando
    CLR     P0.1 ;desabilita escrita
    RET

;-----
; SUBROTINA PARA ESCREVER CHARACTER ASCII NO DISPLAY -
;-----
; ENTRADA: B --> caractere (em ASCII) a ser escrito no display -
; SAIDA: nenhuma -
;
; DESTROI: R7 -
; CHAMA: runT0 -
;-----

PRINTASCII:
    MOV     P1,B
    SETB    P0.0 ;seleciona RS = 1 (dado)
    SETB    P0.1 ;habilita escrita
    MOV     R7,#20 ;aguarda 43us para...
    DJNZ    R7,$ ;... realizar a escrita
    CLR     P0.1 ;desabilita escrita

```


RET

```

;-----
; SUBROTINA PARA ESCREVER DIGITO BCD MENOS SIGNIFICATIVO -
;-----
; Serao escritos caracteres indesejados se o numero nao estiver no formato BCD-
;-----
; ENTRADA: B --> numero BCD -
; SAIDA: nenhuma -
; -
; DESTROI: A -
; CHAMA: PRINTASCII -
;-----

```

PRINTBCDL:

```

PUSH B ;armazena conteudo de B na pilha
MOV A,B
ANL A,#00001111b ;usa apenas 4bits menos significativo
ADD A,#30h ;converte resultado para ASCII
MOV B,A
LCALL PRINTASCII
POP B ;restaura conteudo de B da pilha
RET

```

```

;-----
; SUBROTINA PARA ESCREVER DOIS DIGITOS BCD -
;-----
; Serao escritos caracteres indesejados se o numero nao estiver no formato BCD-
;-----
; ENTRADA: B --> numero BCD -
; SAIDA: nenhuma -
; -
; DESTROI: A, B -
; CHAMA: PRINTASCII -
;-----

```

PRINTBCDHL:

```

PUSH B ;armazena conteudo de B na pilha
MOV A,B
ANL A,#11110000b ;separa 4 bits mais significativos
SWAP A ;troca nibble menos significativo por mais significativo
ADD A,#30h ;converte para ASCII
MOV B,A ;restaura conteudo de B da pilha
LCALL PRINTASCII
POP B
MOV A,B
ANL A,#00001111b ;separa 4 bits menos significativos
ADD A,#30h ;converte para ASCII
MOV B,A
LCALL PRINTASCII
RET

```

```

;-----
; SUBROTINA PARA ESCREVER DIGITO HEXADECIMAL MENOS SIGNIFICATIVO -
;-----
; ENTRADA: B --> numero HEXA -
; SAIDA: nenhuma -
; -
; DESTROI: A, DPTR -
; CHAMA: PRINTASCII -
;-----

```

PRINTHEXAL:

```

PUSH B ;armazena conteudo de B na pilha
MOV DPTR,#TABHEXA
MOV A,B
ANL A,#00001111b ;separa nibble menos significativo
MOVC A,@A+DPTR ;converte caractere para ASCII
MOV B,A
LCALL PRINTASCII
POP B ;restaura conteudo de B da pilha
RET

```

```

;-----
; SUBROTINA PARA ESCREVER DOIS DIGITOS HEXADECIMAIS
;-----
; ENTRADA: B --> numero HEXA
; SAIDA: nenhuma
;
; DESTROI: A, DPTR
; CHAMA: PRINTASCII
;-----

PRINTEXAHL:
    PUSH B ;armazena conteudo de B na pilha
    MOV DPTR,#TABHEXA
    MOV A,B
    ANL A,#11110000b ;separa nibble mais significativo
    SWAP A ;troca nibble menos significativo por mais significativo
    MOVC A,@A+DPTR ;converte caractere para ASCII
    MOV B,A
    LCALL PRINTASCII
    POP B ;restaura conteudo de B da pilha
    MOV A,B
    ANL A,#00001111b ;separa nibble menos significativo
    MOVC A,@A+DPTR ;converte caractere para ASCII
    MOV B,A
    LCALL PRINTASCII
    RET

TABHEXA:
    DB '0123456789ABCDEF'

;-----
; SUBROTINA PARA ESCREVER FRASE EM APENAS UMA LINHA DO DISPLAY
;-----
; ENTRADA: DPTR --> frase a ser escrita
; SAIDA: nenhuma
;
; DESTROI: A, R0
; CHAMA: PRINTASCII
;-----
; Caractere especial: '\' -> fim da frase
;
; Nao verifica se a frase cabe no display
; Precisa ser colocado o caractere '\' no final da frase
;-----

PRINTSTRING:
    PUSH B ;armazena conteudo de B na pilha
    MOV R0,#0 ;inicia a escrita a partir da primeira letra da frase

PRINTS:
    MOV A,R0 ;converte o caractere hexadecimal...
    MOVC A,@A+DPTR ;... em ASCII
    MOV B,A

    CJNE A,#'\',WRITE ;encontrou o caractere '\' ?
    POP B ;sim -> para de escrever
    RET

WRITE:
    LCALL PRINTASCII
    INC R0
    JMP PRINTS

;-----
; SUBROTINA PARA ESCREVER FRASES NAS DUAS LINHAS DO DISPLAY
;-----
; ENTRADA: DPTR --> frase a ser escrita
; SAIDA: nenhuma
;
; DESTROI: A, R0
; CHAMA: PRINTASCII
;-----

```

```

; Caracteres especiais: '/' -> quebra de linha (da primeira para a segunda) -
;                               '\' -> fim da frase -
; -
; Nao verifica se a frase cabe no display -
; Precisa ser colocado o caractere '\' no final da frase -
;-----

PRINTSTRINGALL:
    PUSH    B ;armazena conteudo de B na pilha
    LCALL   CLEARLCD
    MOV     R0,#0 ;inicia a escrita a partir da primeira letra da frase

PRINTSALL:
    MOV     A,R0 ;converte o caractere hexadecimal...
    MOVC    A,@A+DPTR ;... em ASCII
    MOV     B,A

    CJNE    A,'#/',FIMSTR ;encontrou o caractere '/' ?
    MOV     B,#40h ;sim -> começa a escrever na segunda linha
    LCALL   POSCURSOR
    INC     R0
    JMP     PRINTSALL

WRITEALL:
    LCALL   PRINTASCII
    INC     R0
    JMP     PRINTSALL

FIMSTR:
    CJNE    A,'#\\',WRITEALL ;encontrou o caractere '\\'?
    POP     B ;sim -> para de escrever
    RET

;-----
; SUBROTINA PARA DESLOCAR A FRASE A DIREITA -
;-----
; Desloca para a direita todos os caracteres ja escritos no display -
;-----
; ENTRADA: nenhuma -
; SAIDA: nenhuma -
; -
; DESTROI: -
; CHAMA: IWRITE -
;-----

SHIFTLCDR:
    PUSH    B ;armazena conteudo de B na pilha
    MOV     B,#00011100b
    LCALL   IWRITE
    POP     B ;restaura conteudo de B na pilha
    RET

;-----
; SUBROTINA PARA DESLOCAR A FRASE A ESQUERDA -
;-----
; Desloca para a esquerda todos os caracteres ja escritos no display -
;-----
; ENTRADA: nenhuma -
; SAIDA: nenhuma -
; -
; DESTROI: B -
; CHAMA: IWRITE -
;-----

SHIFTLCDL:
    PUSH    B ;armazena conteudo de B na pilha
    MOV     B,#00011000b
    LCALL   IWRITE
    POP     B ;restaura conteudo de B na pilha
    RET

```

```
;-----  
; SUBROTINA PARA GERAR ATRASO DE R1 x 0,5ms -  
;-----  
; ENTRADA: R6 --> valor a ser multiplicado pela base de tempo -  
; -  
; SAIDA: nenhuma -  
; -  
; DESTROI: R7, R6 -  
;-----
```

DELAY:

```
MOV    R7,#228  
DJNZ   R7,$  
DJNZ   R6,DELAY  
RET  
  
END
```